Practical 11: MiniBank Object Serialization, Audit Logs & NIO Summary Report

Problem Definition:

Implement the persistence and auditing layer for MiniBank. Make Account and Customer Serializable with serialVersionUID, create StatePersister to save and load accounts via Object streams, TransactionLog to write append-only transaction logs using NIO, and ReportGenerator to traverse log entries using Files.newDirectoryStream to compute cumulative financial totals.

Code:

```
import java.io.BufferedReader;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.Collections;

class Customer implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private String customerId;

    public Customer(String name, String customerId) {
        this.name = name;
        this.customerId = customerId;
    }

    public String getName() {
        return name;
    }

    public String getCustomerId() {
        return customerId;
    }

    public String toString() {
        return "Customer[id=" + customerId + ", name=" + name + "]";
    }
}

class Account implements Serializable {
    private static final long serialVersionUID = 1L;
    private final String accountNumber;
    private Customer customer;
    protected double balance;
```

```
    public Account(String accountNumber, Customer customer, double initialBalance) {
        this.accountNumber = accountNumber;
        this.customer = customer;
        this.balance = initialBalance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public Customer getCustomer() {
        return customer;
    }

    public String toString() {
        return "Account[" + accountNumber + " | Owner: " + customer.getName() + " | Balance: Rs.
" + balance + "]";
    }
}

class StatePersister {
    public static void save(Account[] accounts, Path file) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(Files.newOutputStream(file))) {
            oos.writeObject(accounts);
        }
    }

    public static Account[] load(Path file) throws IOException, ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(Files.newInputStream(file))) {
            return (Account[]) ois.readObject();
        }
    }
}

class TransactionLog {
    public static void append(Path file, String line) throws IOException {
        if (file.getParent() != null && !Files.exists(file.getParent())) {
            Files.createDirectories(file.getParent());
        }
        Files.write(file, Collections.singletonList(line),
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }
}

class ReportGenerator {
    public static void generateReport(Path logsDir, Path reportFile) throws IOException {
        long totalDeposits = 0;
        long totalWithdrawals = 0;

        try (DirectoryStream<Path> stream = Files.newDirectoryStream(logsDir, "*.log")) {
            for (Path entry : stream) {
                BasicFileAttributes attrs = Files.readAttributes(entry,
BasicFileAttributes.class);
                try (BufferedReader reader = Files.newBufferedReader(entry)) {
```

```
                    String line;
                    while ((line = reader.readLine()) != null) {
                        String[] parts = line.split("\\s+");
                        if (parts.length >= 3) {
                            String type = parts[0];
                            long amount = Long.parseLong(parts[2]);
                            if ("DEPOSIT".equalsIgnoreCase(type)) {
                                totalDeposits += amount;
                            } else if ("WITHDRAW".equalsIgnoreCase(type)) {
                                totalWithdrawals += amount;
                            }
                        }
                    }
                }
            }
        }

        long netChange = totalDeposits - totalWithdrawals;
        String summary = String.format("End-Of-Day Financial Summary Report\nTotal Deposits: Rs.
%d\nTotal Withdrawals: Rs. %d\nNet Liquidity Change: Rs. %d\n",
                totalDeposits, totalWithdrawals, netChange);

        Files.writeString(reportFile, summary, StandardOpenOption.CREATE,
StandardOpenOption.TRUNCATE_EXISTING);
        System.out.print(summary);
    }
}

public class PartB {
    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 11 - Part B: MiniBank Persistence, Audit Log & NIO
Report");
        System.out.println();

        Path accountsPath = Paths.get("data/accounts.dat");
        Path logsDir = Paths.get("data/logs");
        Path dailyLog = logsDir.resolve("daily_transactions.log");
        Path reportFile = Paths.get("data/eod_report.txt");

        try {
            if (accountsPath.getParent() != null && !Files.exists(accountsPath.getParent())) {
                Files.createDirectories(accountsPath.getParent());
            }

            Customer c1 = new Customer("Rudra Modi", "CUST101");
            Customer c2 = new Customer("Aarav Patel", "CUST102");
            Customer c3 = new Customer("Diya Shah", "CUST103");

            Account[] accounts = {
                new Account("AC1001", c1, 15000.0),
                new Account("AC1002", c2, 28000.0),
                new Account("AC1003", c3, 5400.0)
            };

            System.out.println("Saving accounts to binary storage: " + accountsPath);
            StatePersister.save(accounts, accountsPath);
```

```
            System.out.println("Reloading accounts via StatePersister.load():");
            Account[] reloaded = StatePersister.load(accountsPath);
            for (Account acc : reloaded) {
                System.out.println("  " + acc);
            }

            System.out.println();
            System.out.println("Appending transaction events to audit log...");
            TransactionLog.append(dailyLog, "DEPOSIT AC1001 5000");
            TransactionLog.append(dailyLog, "WITHDRAW AC1002 3000");
            TransactionLog.append(dailyLog, "DEPOSIT AC1003 12000");
            TransactionLog.append(dailyLog, "WITHDRAW AC1001 1500");
            System.out.println("Audit logs recorded successfully in " + dailyLog);

            System.out.println();
            System.out.println("Generating reconciliation report with NIO DirectoryStream:");
            ReportGenerator.generateReport(logsDir, reportFile);

        } catch (Exception e) {
            System.err.println("Execution error: " + e.getMessage());
        }
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-11\PartB.java
RUDRA MODI || 25DCS058

Practical 11 - Part B: MiniBank Persistence, Audit Log & NIO Report

Saving accounts to binary storage: data\accounts.dat
Reloading accounts via StatePersister.load():
  Account[AC1001 | Owner: Rudra Modi | Balance: Rs. 15000.0]
  Account[AC1002 | Owner: Aarav Patel | Balance: Rs. 28000.0]
  Account[AC1003 | Owner: Diya Shah | Balance: Rs. 5400.0]

Appending transaction events to audit log...
Audit logs recorded successfully in data\logs\daily_transactions.log

Generating reconciliation report with NIO DirectoryStream:
End-Of-Day Financial Summary Report
Total Deposits: Rs. 85000
Total Withdrawals: Rs. 22500
Net Liquidity Change: Rs. 62500
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: Why is StatePersister implemented with try-with-resources and Object streams?

Answer: StatePersister uses ObjectOutputStream and ObjectInputStream to serialize and deserialize entire Account arrays in binary form. Wrapping these streams in try-with-resources ensures the underlying file descriptors and buffers are deterministically flushed and closed upon completion, even in the event of an I/O error or corruption, preventing file lock leaks and resource exhaustion.

Q2: How does the append-only TransactionLog and NIO ReportGenerator ensure financial auditability?

Answer: The TransactionLog uses Files.write with StandardOpenOption.CREATE and StandardOpenOption.APPEND to persist an immutable, chronological ledger of all banking transactions without overwriting prior records. ReportGenerator lazily traverses the logs folder using Files.newDirectoryStream, reading audit records to calculate cumulative deposits, withdrawals, and net liquid balance, providing verifiable end-of-day reconciliation.